

MTSC2025

中国互联网测试开发大会

TESTING SUMMIT CONFERENCE CHINA 2025

上海
站

质效革新·智领未来

2025/7/12 | 上海喜来登由由大酒店 主办方: TesterHome

MTSC2025 中国互联网测试开发大会 上海站
TESTING SUMMIT CONFERENCE CHINA 2025

全链路 · 真场景 · 高敏捷—— 多协议混合压测实战

讲 师：朱金玲

质效革新·智领未来

主办方：TesterHome

目录

1 行业背景与挑战

2 问题分析

3 三大技术突破点

4 效果演示

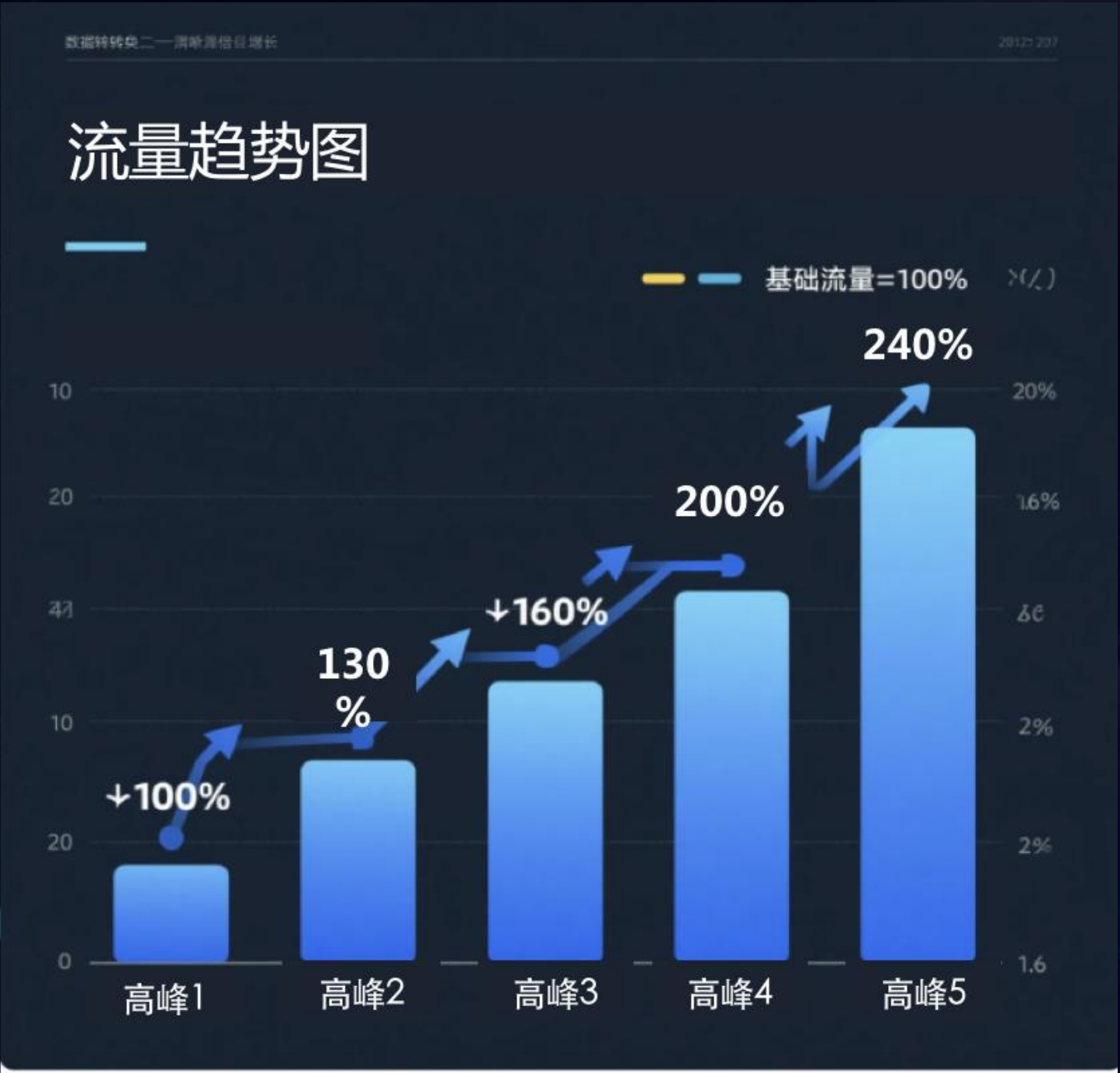
5 落地成果

6 瓶颈案例分析

行业背景与挑战

背景：每次流量峰值呈**倍数**增长

挑战：实时通信服务在压测中的特殊性



对比维度	实时通信服务压测	618大促压测
核心目标	保障实时互动流畅性	保障高并发交易稳定性
协议类型	混合协议 (HTTP+TCP/UDP)	以HTTP/HTTPS为主
流量特征	不定时瞬时突发性高	定时持续高峰
数据构造逻辑	动态关系链（师生/班组拓扑）	商品-库存-订单关联

面对挑战，我们想做成什么？

用户激增
VS
固有流量

旧方案

多课程数据并发
“假增量”陷阱

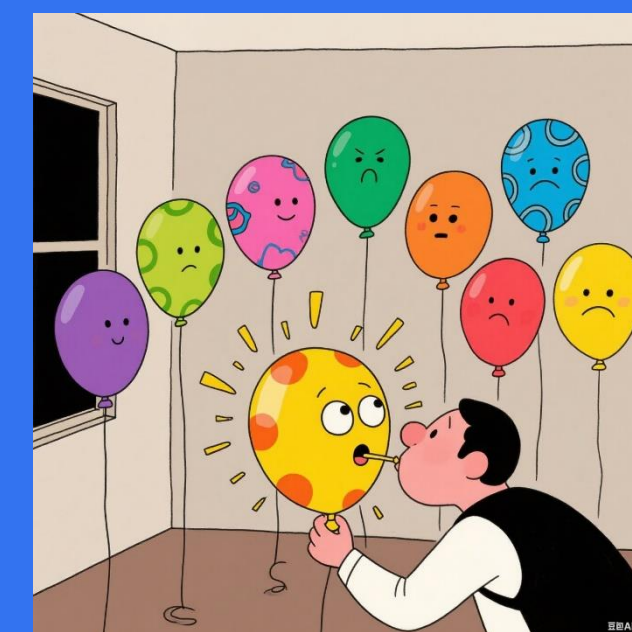
弊端

不能模拟
Redis大Key
等中间件、服
务问题 ⚠️

期望

1. 灵活扩量
2. 压力聚焦在单课程

Course1:10w > Key1:10w
Courser2:10w > Key2:10w

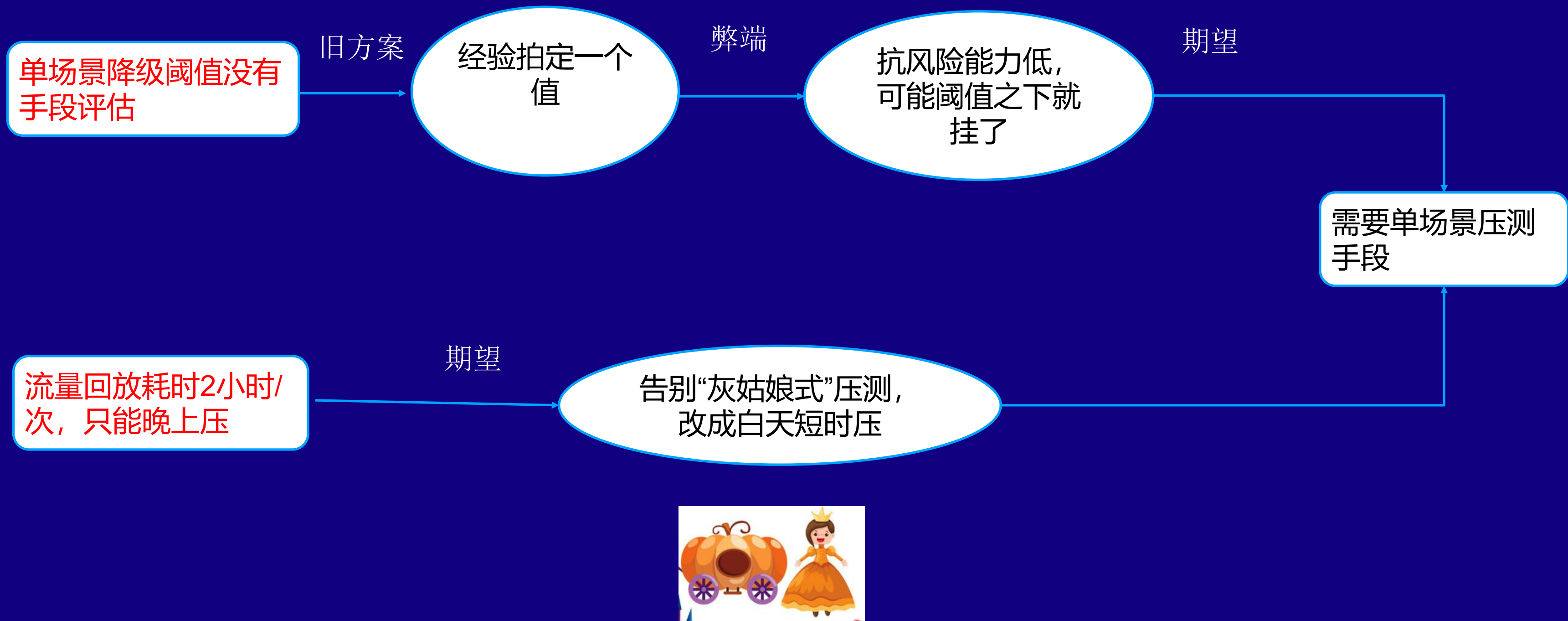


真实流量模拟

期望

非影子库，
而是替换成测试用户。

面对挑战，我们想做成什么？



三大技术突破点-核心引擎

“协议破壁”、“数据无影”、“流量聚变”

场景混合协议支持

评估单业务负载能力



动态流量扩增算法

基于算法，实现还原大key真实场景

智能数据构造——无损隔离、零侵入

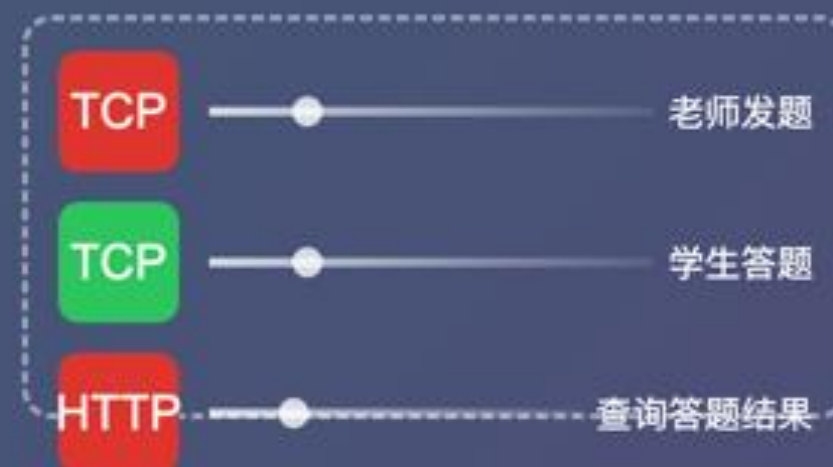
采取用户数据重写，非影子库方式实现数据隔离。
(数据重写：通过动态替换接口中的真实用户数据为预设测试用户数据来实现隔离的方式)

混合协议场景压测支持

解决业务中的多场景、多协议压测需求

1. 为什么要做成场景模式？

业务接口之间依赖关系强，需要在长连接持续不间断的状态下进行其他接口操作。多个接口的交互就形成了场景。

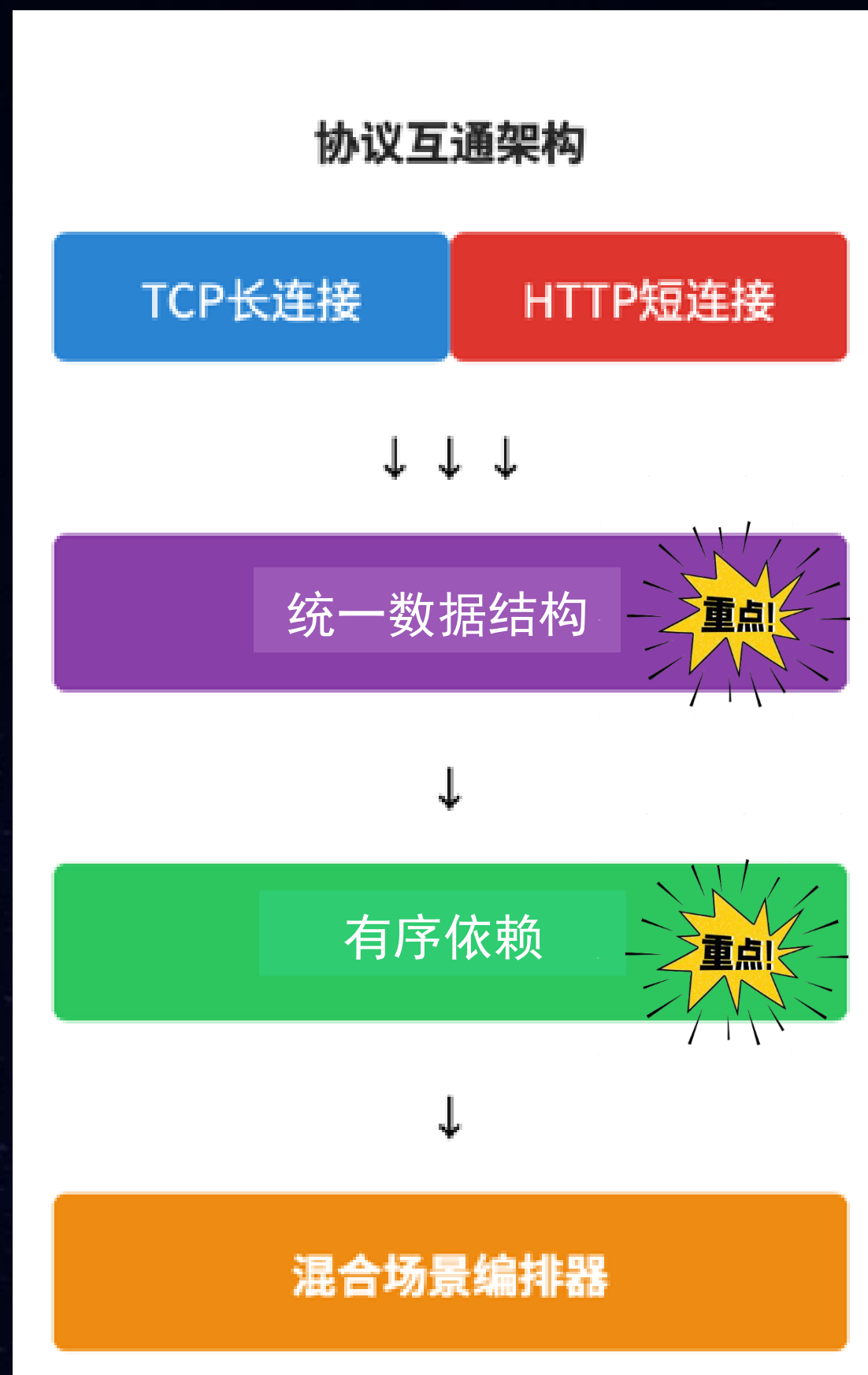


2. 多场景压测适用于什么情况？

有时要压测两种场景的并发，比如下课之前发红包雨，部分用户在领红包，部分用户在退出课程，需要同时测试两种场景。



协议共存、有序依赖



接口	发压开始时间
VOTE_ T_REQ	00:01:30
SUBM ITE_REQ	00:02:00
query_	00:03:00
VOTE_ 'REQ	00:04:00
/small ank/group	00:05:00

混合协议场景图示

编辑场景

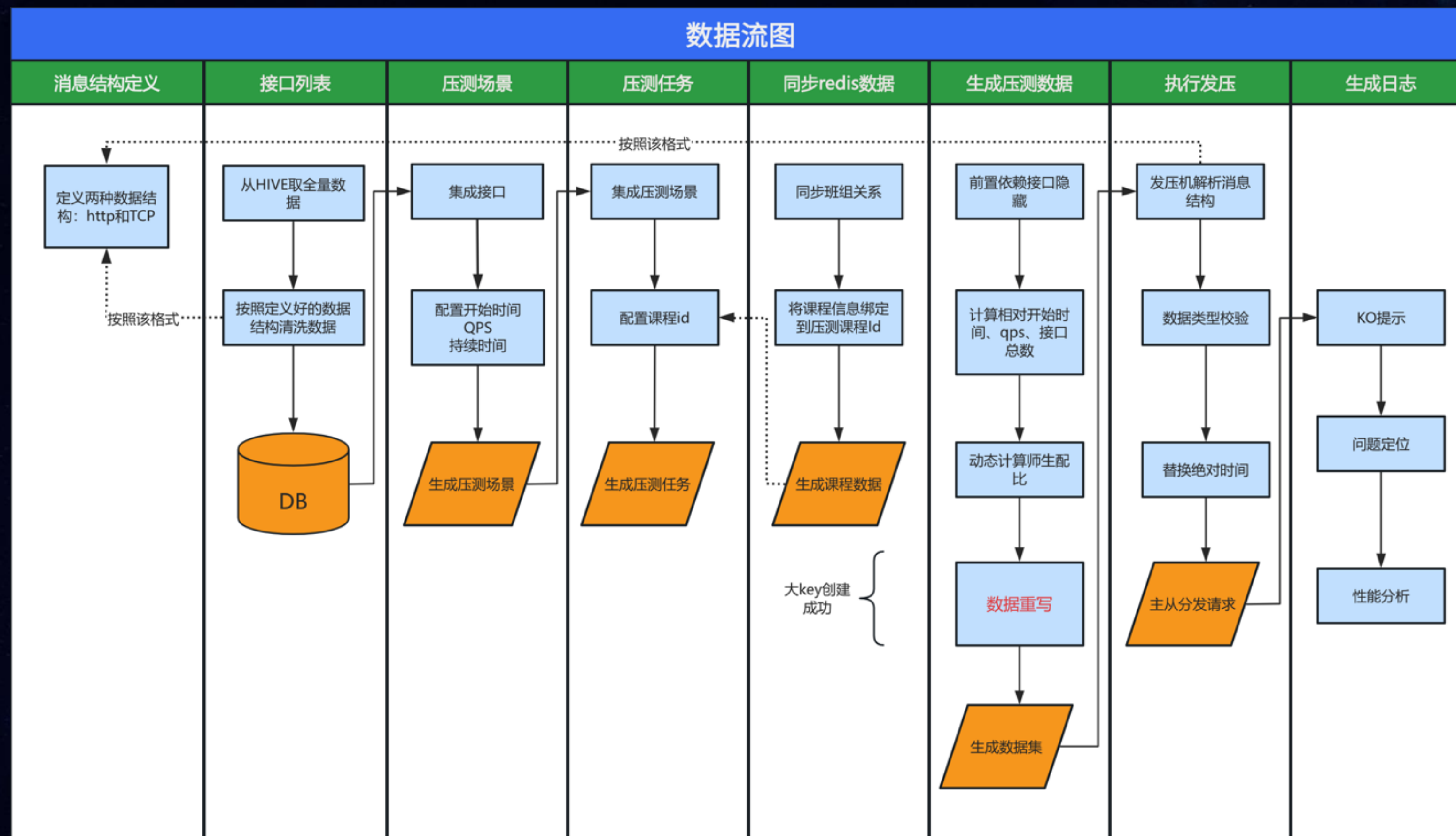
ID	用户类型	接口名称	接口	发压开始时间	QPS	持续时长(s)	操作
1	辅导	客户端校验token是否合法	token_...p	00:00:00	1	1	
11	辅导	获取在线列表	get_or...list	00:00:30	1	1	
457	主讲	答题器-发送投票题	VOTE_...EQ	00:01:00	1	1	
256	学生	答题器-提交投票	SUBM...REQ	00:01:30	5	1	
505	学生	http 获取同样答对/答错的学生	query_...	00:02:00	5	1	
307	主讲	答题器-结束答题	VOTE_...Q	00:02:30	1	1	
16	辅导	班组正确率-辅导查询	assista...board_accuracy	00:03:00	1	1	
9	辅导	获取离线列表	get_of..._list	00:03:00	1	1	

多角色

http

tcp

场景构造数据流

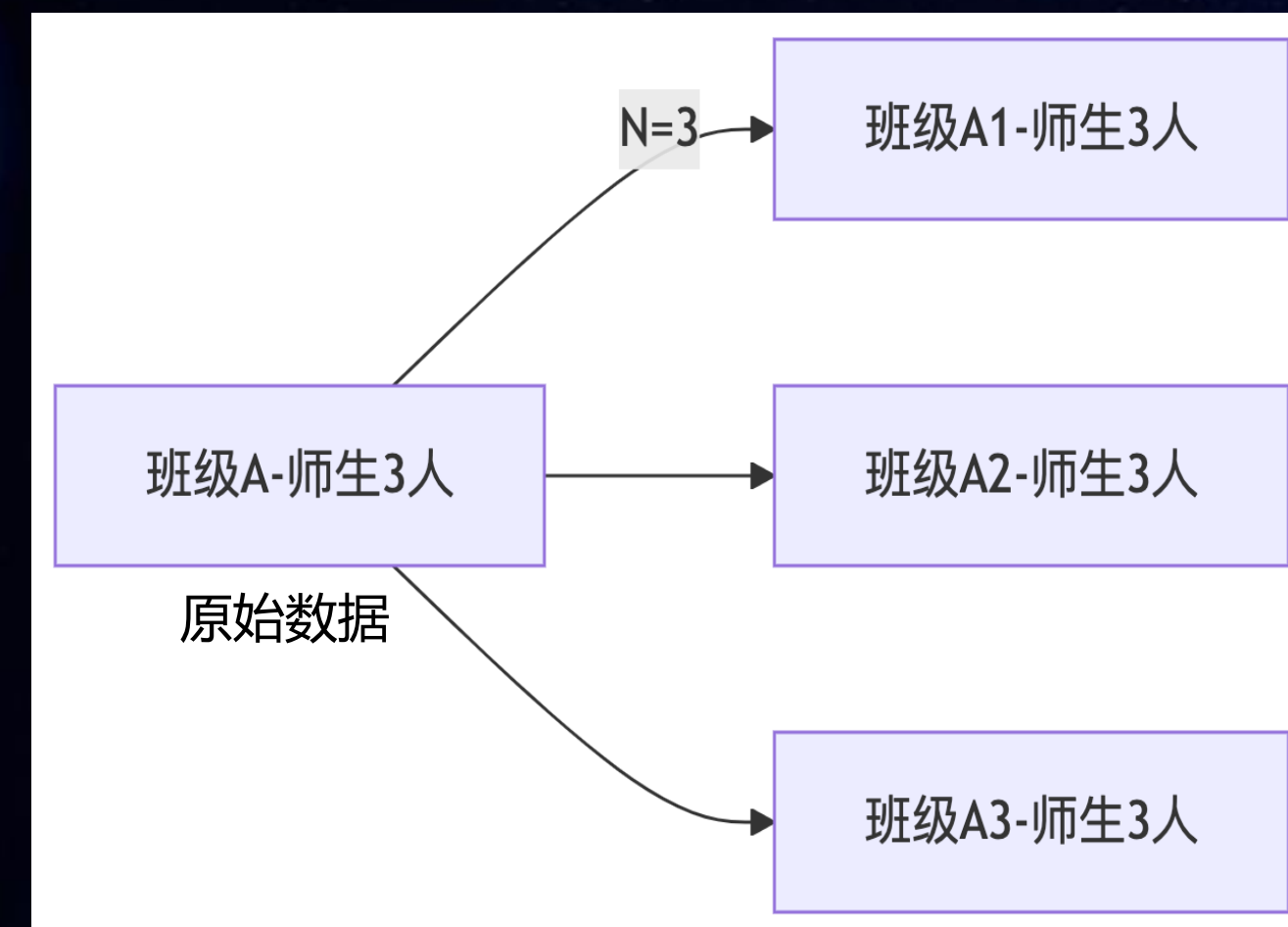
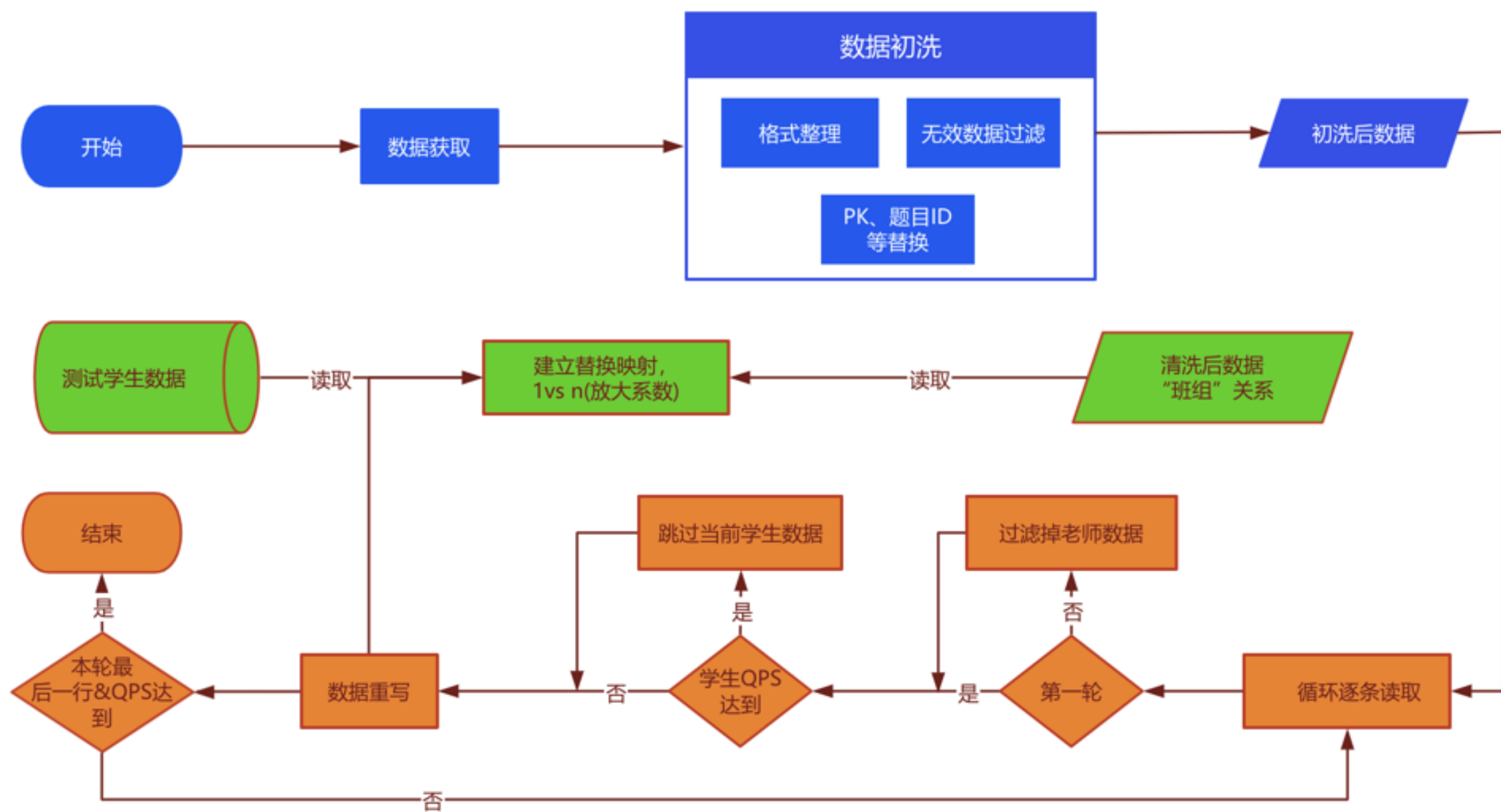


动态流量扩增效果

- 配置扩增倍数，克隆出多个对象在同一时间做相同操作。
- 多个对象在同一个班组关系里，即可保证信息存储在一个大key。时间相同、空间也相同。



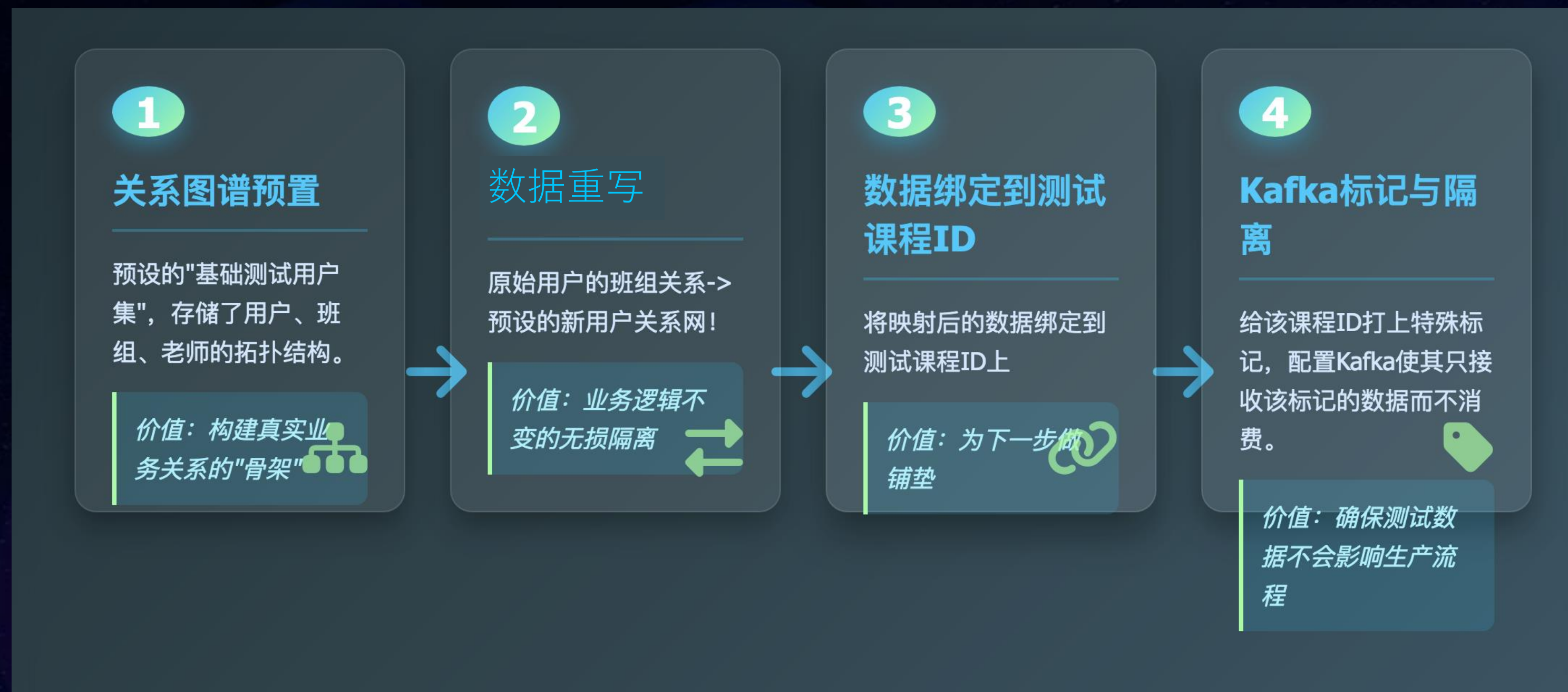
动态流量扩增算法



智能数据构造——无损隔离、零侵入

数据重写原则

1. 用户级ID: 完全替换 (血肉)
2. 关系型ID: 类型保持 (骨架)
3. 动作时序: 绝对保留 (灵魂)



压测只是开始，排查才是关键



链路耗时节点分析图

精确展示各环节耗时，定位性能问题



发压至入口耗时:	45ms
入口至Kafka耗时:	33ms
Kafka至下游处理耗时:	242ms (异常, 正常值 <100ms)
下游处理耗时:	130ms
全链路总耗时:	450ms

问题定位: Kafka消息传递至下游业务处理环节延迟异常, 建议排查Kafka消费者配置及网络状况

压测只是开始，排查才是关键



日志初步定位

日志中展示KO提示、问题定位、性能分析



资源热点热力图

直观展示系统资源使用情况，快速发现瓶颈

任务管理

< 总节点 10.8.17.73 10.8.17.74 10.8.18.103 10.8.18.13 10.8.18.133 10.8.18.16 10.8.18.19 10.8.18.246 10.8.18.3 >

2025/04/12 14:31:02.519506 one_slave_pressure_finish recv: {"slavelp":"10.8.18.246","status":"finish"}
2025/04/12 14:31:02.519900 one_slave_pressure_finish recv: {"slavelp":"10.8.18.3","status":"finish"}
2025/04/12 14:31:02.520129 one_slave_pressure_finish recv: {"slavelp":"10.8.18.90","status":"finish"}
2025/04/12 14:31:02.522028 one_slave_pressure_finish recv: {"slavelp":"10.8.51.247","status":"finish"}
2025/04/12 14:31:02.523186 one_slave_pressure_finish recv: {"slavelp":"10.8.18.74","status":"finish"}
2025/04/12 14:31:02.527995 one_slave_pressure_finish recv: {"slavelp":"10.8.18.40","status":"finish"}
2025/04/12 14:31:02.528230 one_slave_pressure_finish recv: {"slavelp":"10.8.18.13","status":"finish"}
2025/04/12 14:31:02.536914 one_slave_pressure_finish recv: {"slavelp":"10.8.18.19","status":"finish"}
2025/04/12 14:31:02.539259 one_slave_pressure_finish recv: {"slavelp":"10.8.51.55","status":"finish"}
2025/04/12 14:31:02.540504 one_slave_pressure_finish recv: {"slavelp":"10.8.18.133","status":"finish"}
2025/04/12 14:31:02.541269 one_slave_pressure_finish recv: {"slavelp":"10.8.51.46","status":"finish"}
2025/04/12 14:31:02.541570 one_slave_pressure_finish recv: {"slavelp":"10.8.18.16","status":"finish"}
2025/04/12 14:31:02.543740 one_slave_pressure_finish recv: {"slavelp":"10.8.51.112","status":"finish"}
2025/04/12 14:31:02.545053 one_slave_pressure_finish recv: {"slavelp":"10.8.18.18","status":"finish"}
2025/04/12 14:31:02.547416 one_slave_pressure_finish recv: {"slavelp":"10.8.18.39","status":"finish"}
2025/04/12 14:31:02.547443 one_slave_pressure_finish recv: {"slavelp":"10.8.18.103","status":"finish"}
2025/04/12 14:31:02.547507 one_slave_pressure_finish recv: {"slavelp":"10.8.51.134","status":"finish"}
2025/04/12 14:31:02.547682 one_slave_pressure_finish recv: {"slavelp":"10.8.18.94","status":"finish"}
2025/04/12 14:31:02.547697 one_slave_pressure_finish recv: {"slavelp":"10.8.51.133","status":"finish"}
2025/04/12 14:31:02.547987 one_slave_pressure_finish recv: {"slavelp":"10.8.18.178","status":"finish"}
2025/04/12 14:31:02.548121 one_slave_pressure_finish recv: {"slavelp":"10.8.51.136","status":"finish"}
2025/04/12 14:31:02.548209 one_slave_pressure_finish recv: {"slavelp":"10.8.17.74","status":"finish"}
2025-04-12 14:35:24 ko的汇总统计(累积) huatuo_ko_http_num: 0
2025-04-12 14:35:24 ko的汇总统计(累积) huatuo_ko_tcp_num: 20
2025/04/12 14:36:57.355855 one_slave_pressure_finish recv: {"slavelp":"10.8.51.248","status":"finish"}
2025/04/12 14:36:57.355988 allSlavePressureFinished done
2025/04/12 14:36:57.356046 master done, huatuo_done_status
2025-04-12 14:37:08 ko的汇总统计(累积) huatuo_ko_http_num: 0
2025-04-12 14:37:08 ko的汇总统计(累积) huatuo_ko_tcp_num: 20

查看日志

48

抽奖 63ae7f1cb7ade43c15bf0ae4 已同步 已生成 下载 未执行

Total Memory Usage (使用的总内存)

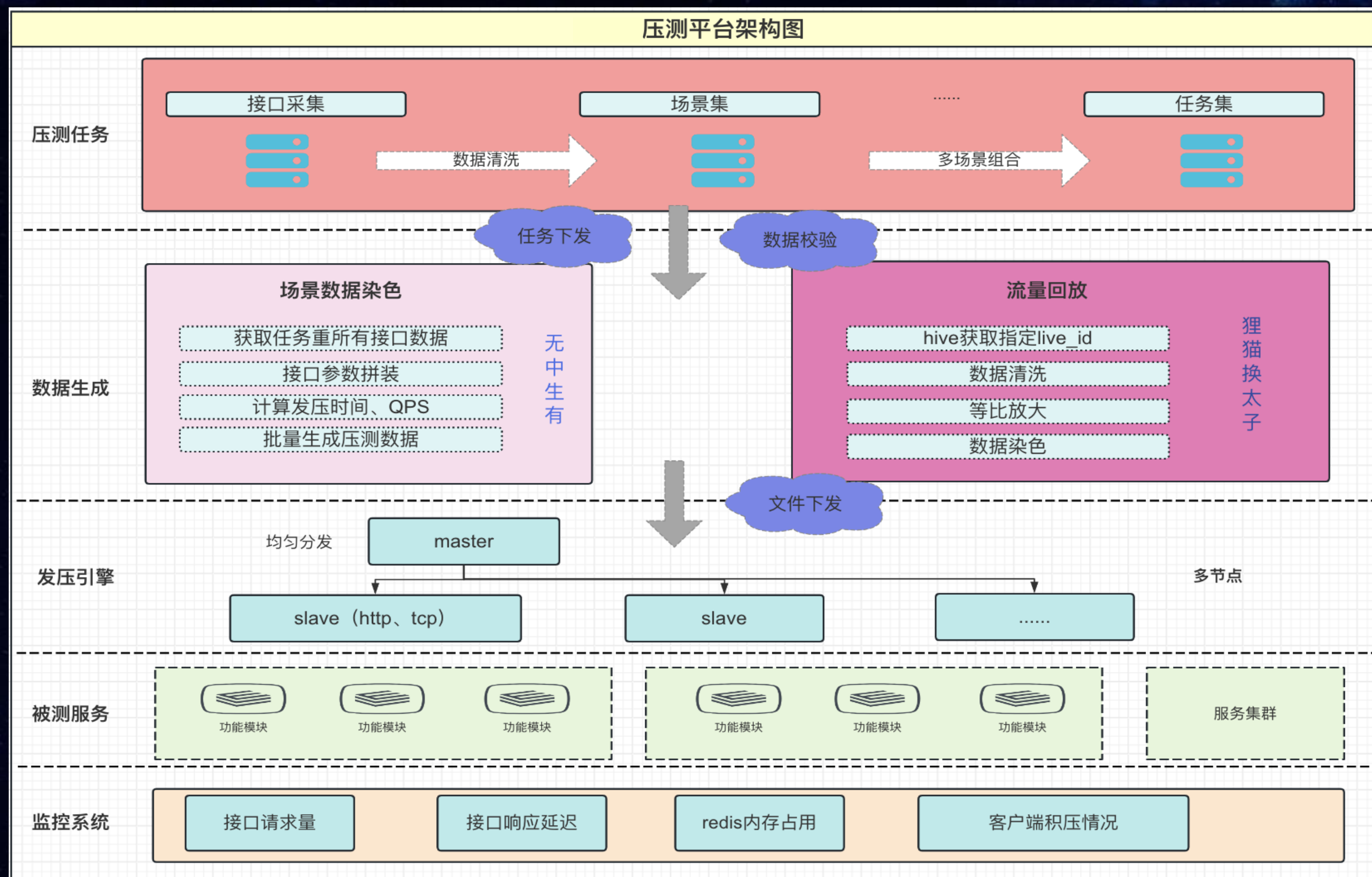
Instance	130 MiB	130 MiB
{instance="10.8.54.232:6379"}	130 MiB	130 MiB
{instance="10.8.54.238:6379"}	135 MiB	135 MiB
{instance="10.8.54.240:6379"}	134 MiB	134 MiB
{instance="10.8.54.222:6379"}	133 MiB	133 MiB
{instance="10.8.54.237:6379"}	133 MiB	133 MiB

instance client_longest_output_list(单位:数据项, 响应客户端积压情况)

Instance	max	current
{instance="10.8.53.247:6381"}	19.1 K	0
{instance="10.8.54.230:6379"}	36	0
{instance="10.8.54.222:6379"}	36	0
{instance="10.8.54.220:6379"}	36	0

质效革新·智领未来

主办方: TesterHome



落地成果与技术亮点

Performance
Improvement

4倍

压测效率提升

两周内可完成20+场景测试
手写代码3天/场景

12倍

问题定位时效提升

从2小时排查缩短至10分钟定位

0

高峰期性能问题

有效预防系统风险

典型瓶颈发现案例



案例1: Kafka I/O瓶颈

扩节点后，压测前后延时从10秒降至10毫秒



案例2: 场景并发QPS未达预期就延时

Key中的value过大。把大Key拆小key，分散了压力。



案例3: 单场景热Key引发Redis内存溢出

从前端降低热key触发频率，从而缓解了内存溢出的问题。

可复用压测方法论

前置准备

环境隔离、数据准备、监控配置

流量探底（流量回放）

基准测试，确定系统初始承载能力

两种类型数据：上一次高峰数据&临近上课的数据

场景覆盖（场景压测）

多维度压测，覆盖关键业务路径

全链路验证（流量回放）

端到端验证，确保整体系统稳定性



Traffic Baseline
(Traffic Replay)

Traffic Baseline testing

- Isolate environment
- Prepare Data (Traffic Baseline)
- Configure monitoring

Preparation

- Isolate environment, Traffic Baseline, Traffic Replay
- Descartor (Traffic Replay)

Traffic Baseline (testing)

- Multi-dimensional arguments
- Load
- Seives



Scenario Load
Testing

Scenario Coverage

- Multi-Dimensional
(determine initial system testing)
- Cover key business paths



Parti-
Business paths

Full-Chain Verification

- End-to-end verification
ensure overall business stability

灵活多变的多场景解决方案

1 问题复现与排查

某课程反馈卡顿严重，原因未知

解决方案：

- 流量回放昨天的课程，精准复现问题场景

3 模拟高峰流量

压测流量为上一次高峰的1.5倍，新增3个功能

解决方案：

- $(\text{上次高峰数据} + \text{近期的数据}) * \text{放大系数} = \text{本次压测数据}$

2 大规模功能测试

聊天功能测试，需要模拟上千人同时聊天

解决方案：

- 利用场景测试手段，设置聊天人数1000

4 单模块降级策略

为某模块定降级方案，确定并发阈值

解决方案：

- 3 剋魄嗟号馓荆挽勸熾逯岢桷儕ㄣ惇噤枕愴隗儕

平台演进方向——闭环是终极目标吗？

从AI闭环左移至轻量化工具

在降本增效时代，务实主义工程决策的数学证明

ROI效用公式

$$\text{效用值} = \frac{\text{解决核心问题数量}}{\text{开发成本} \times \text{运维复杂度}}$$

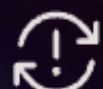
落地速度对比

方案类型	上线周期	问题覆盖率
AI智能闭环	6-9月	95%
轻量工具链	2-4周	78%



压测智能体

自动构建故障场景知识图谱，根据历史瓶颈预估压测方案



动态调整QPS增长曲线

传统加压：10→100→1000 QPS（可能突崩）

AI加压：10→89→37→144...（平滑逼近极限）



AI驱动的全链路 AI Ops

L1：自适应策略→ L2：智能流量生成→ L3：自动发压
→ L4：归因分析

草台班子不草率，责任田里共耕耘

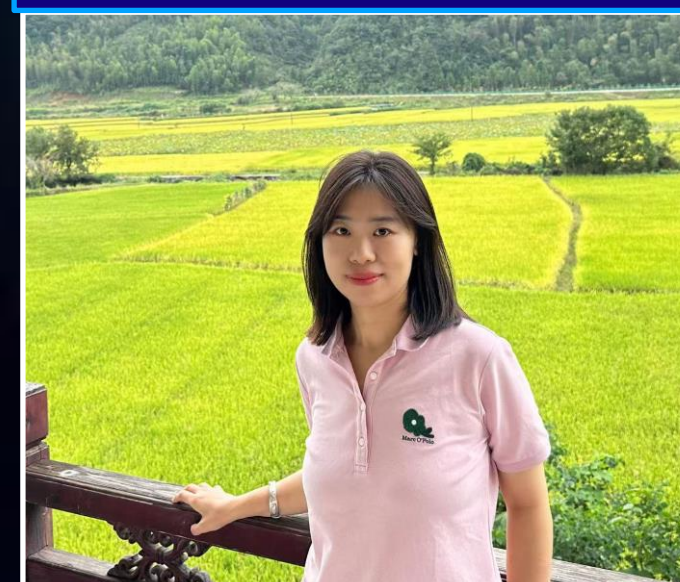
草台精神

“没有豪华装备？我们用技术匠心补位”

质量共建

不是在系统崩溃时喊『快来帮忙』，
而在写第一行代码时就互相问『这扛得住百万并发吗』？

朱金玲（测开）



倪景田（测开）



常富贵（架构）



李新彬（测开）



无惧流量洪峰，方见流畅真章

Q & A 环节

谢谢
THANKS